



Instituto Infnet

Análise e Segurança de Agentes de IA: Projeto de Bloco

Etapa 1

Professor: Tiago Cariolano de Souza Xavier

Email : tiago.xavier@prof.infnet.edu.br

Competências

1. Definir o domínio do sistema com EDA exploratória e API FastAPI autenticada como base
2. Implementar análise estatística completa e controles OWASP Top 10 na API FastAPI auditada por ZAP
3. Consolidar o sistema estatístico e a API segura documentando a transição para a fase preditiva
4. Avaliar a postura de segurança da API adversária com pentest documentado e sistema agêntico completo
5. Defender o sistema de atendimento ao cliente de ponta a ponta com demonstração técnica e pentest adversarial documentado



Avaliação

- 5 Testes de Performance (TP) - **obrigatórios**
 - 1 TP atrasado - limitado a DL
 - 2+ TPs atrasados - limitado a D
- Entrega do Projeto de Bloco - **obrigatório**
- **Presença -> 75% obrigatória**
 - ***Confirmada pelo sistema Zoom e Moodle***



Ferramentas

- Google Colab
- Python3 ou superior
- VSCode



Etapas do Projeto

1. Análise Exploratória de Dados
2. Desenvolvimento de API FASTAPI
3. Desenvolvimento de modelo de IA
4. Integração do modelo de IA na API
5. Ataque de segurança a API
6. Defesa de segurança da API



Formação dos Grupos

- **Tamanho dos grupos:** mínimo de 3, máximo de 4 integrantes
- **Formação do grupo:** enviar e-mail para o professor
 - **Título do e-mail:** Grupo Projeto de Bloco - Análise e Segurança de Agentes de IA
 - **Destinatário:** tiago.xavier@prof.infnet.edu.br e todos os integrantes do grupo
 - **Conteúdo:** nome completo de cada integrante, um por linha
 - ***Se alguma regra não for cumprida, o grupo não é válido***
- **Desfazer/modificar grupo:** enviar e-mail para professor e esperar autorização
- **Data Final:** 05/08/2026



Análise Exploratória de Dados (EDA)



Análise Exploratória dos Dados

- **Passos do EDA:**
 - Compreensão do problema e do dataset
 - Inspeção inicial
 - Verificação da qualidade dos dados
 - Limpeza e preparação dos dados
 - Análise Univariada
 - Análise Multivariada
 - Identificação de Outliers e Anomalias
 - Documentação do EDA e conclusões



Compreensão do problema e dataset

- **Domínio do problema:** definir claramente qual problema se deseja resolver e o que se espera como solução
- **Exemplos:**
 - detectar spam
 - prever doenças
 - reconhecer imagens
 - classificar intenções de clientes
- **Exemplo Domínio de Problema:**
 - *Classificar automaticamente a intenção de uma mensagem enviada por um cliente*
 - **Dataset:** Customer Support Intent
 - **Link:**
 - <https://www.kaggle.com/datasets/scodepy/customer-support-intent-dataset>



Compreensão do problema e dataset

- **Exemplos:**
 - **Mensagem:** "Gostaria de cancelar minha assinatura."
 - **Saída esperada:** Cancelamento
 - **Mensagem:** "Meu boleto venceu."
 - **Saída esperada:** Pagamento
 - **Mensagem:** "Esqueci minha senha."
 - **Saída esperada:** Suporte Técnico



Inspeção Inicial

- **O que deve ser inspecionado?**
 1. Tamanho do dataset
 2. Quais são as variáveis
 3. Significado colunas
 4. Tipos dos dados
 5. Organização dos registros
 6. Exemplos de valores
 7. Identificação de variáveis:
 - 7.1. Numéricas, categóricas, textuais ou temporais
 8. Variáveis relevantes para o objetivo da análise
 9. Possíveis problemas nos dados



Verificação da Qualidade dos Dados

- **O que deve ser verificado?**
 1. Valores ausentes e em qual quantidade;
 2. Registros duplicados;
 3. Tipos dos dados estão corretos;
 4. Categorias escritas de formas diferentes;
 5. Textos vazios ou espaços extras;
 6. Valores impossíveis ou fora do contexto;
 7. Erros de digitação ou preenchimento;
 8. Problemas que precisam ser corrigidos antes da análise.



Limpeza e Preparação dos Dados

- **Principais operações:**
 1. Remover/tratar registros duplicados desnecessários;
 2. Remover/tratar valores ausentes tratados de forma justificada;
 3. Corrigir tipos das colunas;
 4. Corrigir espaços, erros de preenchimento e inconsistências evidentes;
 5. Corrigir valores inválidos.



Análise Univariada

- **Variáveis numéricas:**
 1. média, mediana, mínimo e máximo;
 2. quartis, variância e desvio-padrão;
 3. distribuição dos valores;
- **Variáveis categóricas:**
 1. quantidade de categorias;
 2. frequência absoluta e relativa;
 3. categoria mais frequente;



Análise Multivariada

- **Variáveis numéricas:**

1. Correlação linear entre alvo x independentes
2. Correlação linear entre variáveis independentes
3. Correlação não-linear entre alvo x independentes
4. Correlação não-linear entre variáveis independentes
5. Analisar variáveis preditoras
6. Analisar e tratar variáveis redundantes

- **Variáveis categóricas:**

1. Associação entre alvo x independentes
2. Associação entre variáveis independentes
3. Analisar variáveis preditoras
4. Analisar e tratar variáveis redundantes



Identificar outliers e anomalias

- **Variáveis numéricas:**
 1. Selecionar método de identificação de outlier;
 - 1.1. Q-Q Plot, Skewness, Teste Shapiro-Wilk
 2. Identificar outliers com IRQ ou ZScore
- **Variáveis categóricas:**
 1. Identificar categorias raras;
 2. Identificar possível desequilíbrio entre as classes.



Documentação do EDA e Conclusões

- **Documentação do EDA:** consiste na organização do notebook e das observações no markdown
- Conclusões:
 - **Principais achados:** mostrar as principais descobertas nos dados
 - Desbalanceamento, variáveis mais importantes, valores ausentes, outliers, padrões e possíveis redundâncias, etc.
 - **Operações realizadas:** mostrar o que foi feito no EDA nas fases anteriores
 - Limpeza, remoção de duplicatas, tratamento de valores ausentes, análises estatísticas, correlações, testes de associação e identificação de outliers, etc.



Tarefa em grupo

- **Realizar EDA com dataset Bank Marketing:**
 - O objetivo é prever se um cliente aceitará ou não contratar um depósito bancário, portanto também é um problema de classificação binária.
 - variáveis numéricas, como age, balance e duração do contato;
 - variáveis categóricas, como job, marital, education e housing;
 - valores desconhecidos em algumas categorias;
 - variável-alvo y, com valores yes e no
- **Link dataset Bank Marketing:**
<https://www.kaggle.com/dvaser/bank-marketing>



Aplicação FastAPI

Da instalação à primeira API

```
pip install fastapi  
fastapi dev main.py  
fast
```

Criar aplicação API RESTFUL com fastapi
com GET e POST

O que é uma API?

API — Application Programming Interface permite que diferentes aplicações se comuniquem.



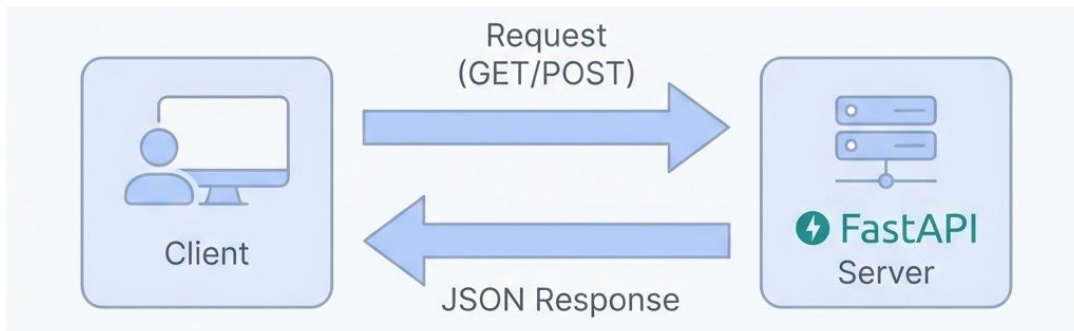
Frontend

- Frontend não precisa saber como os produtos estão armazenados.
- Ele apenas precisa saber como conversar com a API.



Cliente/Servidor

- Cliente solicita:
 - GET /produtos
- Servidor responde:
 - [{ "id": 1, "nome": "Notebook", "preco": 3500 }, { "id": 2, "nome": "Mouse", "preco": 100 }]



Métodos HTTP



GET

- Consultar dados
 - Exemplo: listar produtos



POST

- Criar dados
 - Exemplo: cadastrar produto



PUT

- Atualizar um recurso
 - Exemplo: alterar produto



DELETE

- Excluir um recurso
 - Exemplo: remover produto

Métodos HTTP



GET

- Consultar dados
 - Exemplo: listar produtos



POST

- Criar dados
 - Exemplo: cadastrar produto



PUT

- Atualizar um recurso
 - Exemplo: alterar produto



DELETE

- Excluir um recurso
 - Exemplo: remover produto

Primeira aplicação

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"mensagem": "Minha primeira API"}
```



Importando FastAPI

- Importando a classe principal do framework.



Criando a aplicação

- app representa a aplicação
- Nesse objeto são registrados os endpoints.



Criando uma rota

- `@app.get("/")` informa ao FastAPI:
 - “Quando alguém fizer uma requisição GET para /, execute a função abaixo”
- Decorator associa uma rota e um método HTTP a uma função Python.



Criando a função

- `def home() :`
 - `return {"mensagem": "Minha primeira API"}`
 - Quando o endpoint for acessado, a função será executada.
- `{"mensagem": "Minha primeira API"}`
 - Enviado ao cliente como uma resposta JSON.

Executando a aplicação



No terminal

- `fastapi dev main.py`



No navegador

- `http://127.0.0.1:8000`



Resposta no navegador

- `{ "mensagem": "Minha primeira API" }`



Swagger

- <http://127.0.0.1:8000/docs> abre uma interface de acesso a API chamada swagger

Criando um recurso: produtos

```
produtos = [  
  {  
    "id": 1,  
    "nome": "Notebook",  
    "preco": 3500.0  
  },  
  {  
    "id": 2,  
    "nome": "Mouse",  
    "preco": 100.0  
  }  
]
```



Produtos

- Essa lista funcionará temporariamente como nosso "banco de dados".
- Em uma aplicação real, provavelmente utilizaríamos um banco de dados

Criando um endpoint para listar produtos

```
@app.get("/produtos")
def listar_produtos():
    return produtos
```



GET /produtos ou <http://127.0.0.1:8000/produtos>

- Resultado:
 - [
 - {
 - "id": 1,
 - "nome": "Notebook",
 - "preco": 3500.0
 - },
 - {
 - "id": 2,
 - "nome": "Mouse",
 - "preco": 100.0
 - }
 -]

Parâmetros de rota

```
@app.get("/produtos/{produto_id}")
def buscar_produto(produto_id: int):

    for produto in produtos:
        if produto["id"] == produto_id:
            return produto

    return {"erro": "Produto não encontrado"}
```



GET /produtos/1 ou http://127.0.0.1:8000/produtos/1

- produto_id: int indica que produto_id deve ser um inteiro
- FastAPI reconhece {produto_id} como parâmetro da rota e utiliza a anotação int para converter e validar o valor recebido
- Resultado:
 - {
 "id": 1,
 "nome": "Notebook",
 "preco": 3500.0
}

Retornando erros HTTP

```
@app.get("/produtos/{produto_id}")
def buscar_produto(produto_id: int):

    for produto in produtos:
        if produto["id"] == produto_id:
            return produto

    raise HTTPException(
        status_code=404,
        detail="Produto não encontrado"
    )
```



**GET /produtos/abc ou
http://127.0.0.1:8000/produtos/abc**

- Produto abc não existe
- Agora, quando um produto não existir, a API retorna:
 - 404 Not Found
- Resultado:
 - { "detail": "Produto não encontrado" }

Query parameters

```
@app.get("/listar_produtos")
def listar_produtos(limite: int = 10):
    return produtos[:limite]
```



GET /produtos?limite=1

- Limite não aparece dentro da rota: "/produtos"
 - FastAPI interpreta esse argumento como um query parameter.
- Resultado:
 - [
 - {
 - "id": 1,
 - "nome": "Notebook",
 - "preco": 3500.0

Criando dados com POST

```
@app.post("/produtos", status_code=201)
async def criar_produto(request: Request):
```

```
    produto = await request.json()
```

```
    novo_produto = {
        "id": len(produtos) + 1,
        "nome": produto["nome"],
        "preco": produto["preco"]
    }
    produtos.append(novo_produto)
```

```
    return novo_produto
```



POST /produtos

- Cliente envia:
 - {
 "nome": "Teclado",
 "preco": 250
}
- Cliente recebe:
 - {
 "id": 3,
 "nome": "Teclado",
 "preco": 250
}



Método request.json()

- Lê o corpo da requisição e o converte para um dicionário Python.

Testando o POST com requests

```
import requests

url1 =
"http://127.0.0.1:8000/criar_produto"
url2=
"http://127.0.0.1:8000/listar_produtos"
url3=
"http://127.0.0.1:8000/listar_produtos?lim
ite=4"
novo_produto = {
    "nome": "Teclado",
    "preco": 4000.0
}
response = requests.post(url1,
json=novo_produto)
response = requests.post(url1,
json=novo_produto)
response = requests.get(url3)
print(response.json())
```



Instalar requests

- pip install requests



requests.get e requests.post

- Realizam requisições GET e POST HTTP
 - request.get: apenas url
 - request.post: url e conteúdo no formato json a ser enviado



Outro Terminal

- python test.py
 - Se o arquivo se chamar test.py

Modelos Pydantic

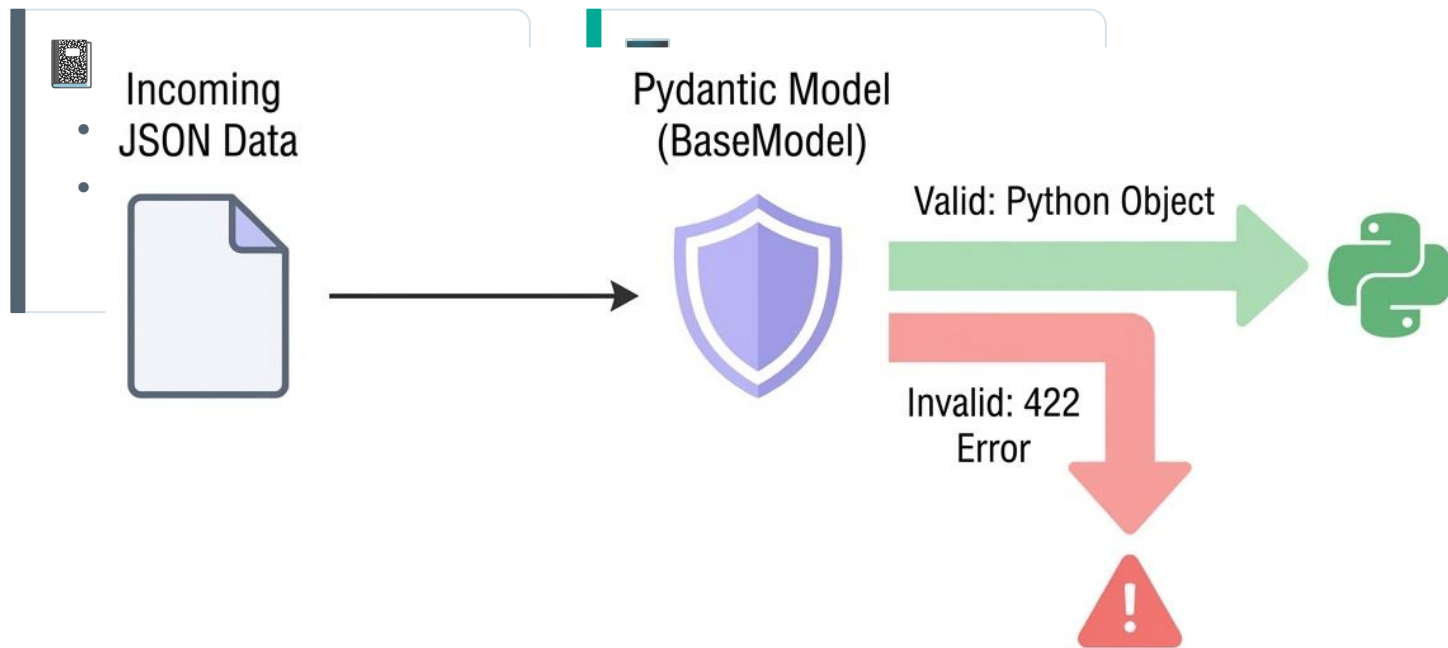
Da instalação à primeira API

```
pip install fastapi  
fastapi dev main.py  
fast
```

Criar aplicação API RESTFUL com fastapi
com GET e POST

Modelos Pydantic

Pydantic: biblioteca Python utilizada para trabalhar com dados estruturados e validação baseada em tipos.



Modelo Pydantic



Definição do modelo

- ```
class Produto(BaseModel):
 nome: str
 preco: float
```



## Atributos esperados

- Um Produto possui:
  - nome → str
  - preco → float

# Importando BaseModel

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Produto(BaseModel):
 nome: str
 preco: float
```



## Produto

- Classe chamada Produto que herda BaseModel
- Formato esperado dos dados:
  - nome: str
  - preco: float
- Produto válido:
  - {  
    "nome": "Notebook",  
    "preco": 3500  
}
- Produto inválido:
  - {  
    "nome": 1000,  
    "preco": 3500  
}

# Criando objetos Pydantic em Python

```
from pydantic import BaseModel
```

```
class Produto(BaseModel):
 nome: str
 preco: float
```

```
produto = Produto(
 nome="Notebook",
 preco=3500
)
```

```
dados = produto.model_dump()

print(dados)
```



## Objeto Pydantic Produto

- Acesso aos seus atributos:
  - `print(produto.nome)`  
`print(produto.preco)`



## `produto.model_dump()`

- Converte um modelo para dicionário
- Resultado:
  - `{  
 "nome": "Notebook",  
 "preco": 3500.0  
}`

# Utilizando o modelo no FastAPI

```
@app.post("/produtos", status_code=201)
def criar_produto(produto: Produto):

 novo_produto = {
 "id": len(produtos) + 1,
 "nome": produto.nome,
 "preco": produto.preco
 }

 produtos.append(novo_produto)

 return novo_produto
```



## produto: Produto

- FastAPI percebe que Produto é um modelo Pydantic
  - Dados devem vir no corpo da requisição e devem possuir a estrutura definida por esse modelo.
- FastAPI realiza esse trabalho automaticamente.

# Validação automática



## Validação do modelo

- Classe:
  - `class Produto(BaseModel):`  
    `nome: str`  
    `preco: float`
- Envio 1:
  - `{`  
    `"nome": "Teclado",`  
    `"preco": "abc"`  
    `}`
- Envio 2:
  - `{`  
    `"nome": "Teclado"`  
    `}`



## Erro retornado

- Envio1:
  - O valor "abc" não pode ser utilizado como um número float
- Envio 2:
  - Os dois campos são obrigatórios
- FastAPI rejeitará automaticamente a requisição antes mesmo da execução da nossa função.

# Adicionando um novo campo

```
class Produto(BaseModel):
 nome: str
 preco: float
 categoria: str
```



## Corpo esperado

- ```
{  
  "nome": "Teclado",  
  "preco": 250,  
  "categoria": "Periféricos"  
}
```
- Alterar modelo já modifica o formato esperado pela API

Campos opcionais e valores padrão

```
class Produto(BaseModel):  
    nome: str  
    preco: float  
    categoria: str  
    descricao: str | None = None  
    disponivel : bool = True
```

JSONs válidos

- {
 "nome": "Teclado",
 "preco": 250,
 "categoria": "Periféricos"
}
- {
 "nome": "Teclado",
 "preco": 250,
 "categoria": "Periféricos",
 "descricao": "Teclado mecânico"
}



Campos opcionais

- | None = None indica que o campo é opcional
- Opcionais:
 - descricao



Campos padrão

- var : type = value indica que value é padrão para var
- Valor padrão é opcional
- Padrão:
 - disponivel = True

Validações com Field

```
from pydantic import BaseModel, Field
class Produto(BaseModel):
    nome: str = Field(min_length=2)
    preco: float = Field(gt=0 )
    categoria: str
    descricao: str | None = None
    disponivel : bool = True
```



Validações de campos

- Field(gt=0) significa greater than 0 ou maior que zero
- Field(min_length=2) significa que o nome precisa ter pelo menos dois caracteres

Dica: Validações com Fields

<https://pydantic.dev/docs/validation/latest/concepts/fields/>

FASTAPI

Autenticação com OAuth2PasswordBearer

Da instalação à primeira API

```
pip install fastapi
fastapi dev main.py
fast
```

Criar aplicação API RESTFUL com
fastapi com GET e POST

Importando OAuth2PasswordBearer

```
from fastapi import
    FastAPI, HTTPException, Depends
from fastapi.security import
    OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme =
OAuth2PasswordBearer(tokenUrl="token")
...
```



oauth2_scheme =
OAuth2PasswordBearer(tokenUrl="token")

- Esquema de autenticação OAuth2 utilizando **Bearer Token**
- Quando utilizarmos esse objeto como uma dependência, FastAPI poderá procurar o token enviado no cabeçalho:
 - **Authorization: Bearer token-fake-admin**
- O parâmetro tokenUrl="token" indica qual endpoint será utilizado pelo cliente para obter o token
- OAuth2PasswordBearer(tokenUrl="token") não cria automaticamente a rota /token.
 - O tokenUrl informa ao esquema OAuth2 e à documentação da API onde o cliente poderá solicitar o token

Criando usuário fake

```
from fastapi import
    FastAPI, HTTPException, Depends
from fastapi.security import
    OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme =
    OAuth2PasswordBearer(tokenUrl="token")

USUARIO_FAKE = {
    "username": "admin",
    "password": "1234"
}

TOKEN_FAKE = "token-fake-admin"
```



Segurança

- Armazenar senha dessa forma é propositalmente inseguro e está sendo feito apenas para que possamos enxergar o fluxo da autenticação sem introduzir hashing ou JWT

Criando a rota de login

```
from fastapi import FastAPI, HTTPException, Request, Depends
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel
from produto import Produto

app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")

USUARIO_FAKE = {"username": "admin", "password": "1234"}
TOKEN_FAKE = "token-fake-admin"

@app.post("/token")
def login(form_data: OAuth2PasswordRequestForm =
Depends(OAuth2PasswordRequestForm)):
    if form_data.username != USUARIO_FAKE["username"] or \
        form_data.password != USUARIO_FAKE["password"]:
        raise HTTPException(status_code=401, detail="Usuário ou
senha inválidos.")
    return {
        "access_token": TOKEN_FAKE,
        "token_type": "Bearer"
    }

...
// Demais rotas
```

form_data: OAuth2PasswordRequestForm
=

Depends(OAuth2PasswordRequestForm)

- FastAPI recebe os dados enviados pelo cliente e disponibiliza:
 - form_data.username
 - form_data.password

var: class = Depends(method)

- Antes de executar a função login, execute a lógica representada por method e coloque o resultado na variável var.

Testando o token

```
@app.get("/teste-token")
def teste_token(token: str = Depends(oauth2_scheme)):
    return {"token_recebido": token}
```



Rota teste-token

- Cliente envia:
 - Authorization: Bearer t
- FastAPI utiliza oauth2_scheme para obter o conteúdo do token
- teste_token recebe
 - token = "token-fake-admin"
- teste-token envia:
 - {
 "token_recebido": "token-fake-admin"
}



Botão Authorize no Swagger

- Swagger cria botão "Authorize" para entrar com username e password

Função de validação de token

```
def validar_token(token: str = Depends(oauth2_scheme)):\n\n    if token != TOKEN_FAKE:\n        raise HTTPException(\n            status_code=401,\n            detail="Token inválido",\n            headers={"WWW-Authenticate": "Bearer"}\n        )\n\n    return token
```



validar_token

- Função para comparar o token recebido pelo usuário com o token real

Protegendo uma rota

```
@app.get("/listar_produtos_autenticado/")
def listar_produtos(limite: int = 10, token: str =
Depends(validar_token)):
    return produtos[:limite]
```



token: str = Depends(validar_token)

- Executa a função `listar_produtos_autenticado` somente após `validar_token`

Segurança com JWT

Da instalação à primeira API

```
pip install fastapi  
fastapi dev main.py  
fast
```

Criar aplicação API RESTFUL com
fastapi com GET e POST

JWT (JSON Web Token)



JWT (JSON Web Token)

- representação compacta de informações, chamadas claims, que pode ser assinada para permitir a verificação de sua integridade



Biblioteca PyJWT

- permite codificar e decodificar JSON Web Tokens em Python



Dinâmico

- O valor real será maior e poderá mudar a cada login dependendo dos dados utilizados para criá-lo.



Verificação

- Ao invés:
 - token-fake-admin
- Tem-se:
 - eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbGlzImV4cCI6MTc... .abc123...



instalação

- `py -m pip install pyjwt`

Importando PyJWT

```
from datetime import datetime, timedelta, timezone
import jwt
from jwt.exceptions import InvalidTokenError

from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel
```



Imports

- `datetime`: usado para registrar o momento de criação ou o limite de validade do token.
- `timedelta`: usado para somar minutos ou dias à data atual (ex: definir que o token dura 30 minutos).
- `timezone`: Gerencia fusos horários, garantindo que o cálculo de expiração do token seja preciso independentemente de onde o servidor esteja hospedado (usando o padrão UTC).
- `InvalidTokenError`: É o aviso de erro disparado automaticamente pelo jwt caso o sistema tente ler um token que foi falsificado ou adulterado.

Criando uma função gerar_token()

```
def gerar_token(username: str):  
    expiracao = datetime.now(timezone.utc) +  
    timedelta(minutes=30)  
  
    dados = {  
        "sub": username,  
        "exp": expiracao  
    }  
  
    token = jwt.encode(  
        dados,  
        SECRET_KEY,  
        algorithm=HS256  
    )  
  
    return token
```



função gerar_token

- A função recebe username
- expiracao é o momento de expiração do token
 - expiração = agora + 30 minutos
- sub (subject): identifica quem é o dono do token (geralmente o username ou ID do usuário)
- exp (expiration): identifica quando o token expira



função jwt.encode(dados, chave, algoritmo)

- dados: o conteúdo do token (payload), tipo sub e exp. Não é secreto, só codificado.
- SECRET_KEY: a chave usada pra assinar o token, garantindo que ninguém adultere o conteúdo sem ser detectado. Fica só no servidor.
- algorithm: qual algoritmo criptográfico gera essa assinatura (ex: "HS256").

Nova rota com JWT

```
@app.post("/token_jwt")
def login(form_data: OAuth2PasswordRequestForm =
Depends(OAuth2PasswordRequestForm )):

    if (
        form_data.username != USUARIO_FAKE["username"]
        or form_data.password !=
USUARIO_FAKE["password"]
    ):
        raise HTTPException(
            status_code=401,
            detail="Usuário ou senha inválidos"
        )

    token = gerar_token(form_data.username)

    return {
        "access_token": token,
        "token_type": "bearer"
    }
```

Função validar_token_jwt()

```
def validar_token_jwt(token: str = Depends(oauth2_scheme)):

    erro = HTTPException(
        status_code=401,
        detail="Token inválido ou expirado",
        headers={"WWW-Authenticate": "Bearer"}
    )

    try:

        payload = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=["HS256"]
        )

        username = payload.get("sub")

        if username is None:
            raise erro

    except InvalidTokenError:
        raise erro

    return username
```